Listen

Learning Doc 07: The Cloud API and Database
Endpoints, PostgreSQL tables, logins and secrets

Detailed design, Section 6

14 October 2026

Abstract

The server is the meeting point between your laptop and the web app. This document designs it: what an API endpoint is, every endpoint Listen needs, the PostgreSQL tables behind them, how you log in with a username and password without the server ever storing your password, how the laptop proves its identity, and how the design copes with a free server that falls asleep.

Contents

1 Decisions this section is built on **2**

2 What an API is, concretely **2**

3 The endpoints **3**
3.1 Logging in (web app) . . . 3
3.2 Devices (web app) . . . 3
3.3 Sync (laptop) . . . 3
3.4 Analytics (web app) . . . 3
3.5 History edits (web app) . . . 3
3.6 Command library (web app) . . . 4
3.7 Endpoints are code you write, not a fixed list . . . 4

4 The PostgreSQL tables **4**

5 The ORM in depth: SQLAlchemy by example **5**
5.1 The problem an ORM solves . . . 5
5.2 Step 1: describe a table once, as a class . . . 5
5.3 Step 2: read, write, change, delete . . . 6
5.4 Step 3: when you need a database-specific feature . . . 7
5.5 What an ORM gives you, and what it costs . . . 7

6 Logging in without storing your password **7**

7 Secrets and configuration **8**

8 Living with a server that sleeps **8**

9 Errors and tests **9**

10 Glossary **9**

11 Check your understanding **10**

1 Decisions this section is built on

Topic	Decision
Budget	Free tiers only. The server may sleep when idle and take 20–50 s to wake.
Web login	Username and password.
AI builder	Runs on the **laptop** with your own API key in the agent's `.env`. The server needs no AI key; the web app edits commands with the guided form.
From earlier	API-first (Doc 01), laptop-master commands and the sync protocol (Doc 06), `user_id` on every row.

2 What an API is, concretely

Concept: HTTP request and response

Every interaction is a **request** from a client and a **response** from the server. A request has a **method** (what kind of action), a **path** (which thing), optional **headers** (extra information, such as who you are) and an optional **body** (data, as JSON). A response has a **status code** (did it work?) and a body.

Method	Meaning (convention)
`GET`	Read something; never changes anything
`POST`	Create something, or perform an action
`PATCH`	Change part of something
`DELETE`	Remove something

Status	Meaning
200 / 201	OK / created
400 / 422	Your request is malformed
401	Who are you? (not logged in)
403	You may not do that
404	No such thing
500	The server broke

Concept: Endpoint

An **endpoint** is one method + path the server answers, such as `GET /analytics/apps`. The full list of endpoints, with what each accepts and returns, *is* the API. FastAPI generates an interactive web page documenting all of them automatically at `/docs`, which is what makes the API usable by others with no extra work.

Concept: Validation with Pydantic

Every request body and response is described by a **Pydantic model**: a Python class with typed fields. FastAPI checks every incoming request against it *before* your code runs: a missing field or a text where a number belongs is rejected with a 422 and a clear message. Your code never sees malformed data.

3 The endpoints

Two kinds of caller, two kinds of proof (Section 6): the **web app** uses a login session; the **laptop** uses a device token.

3.1 Logging in (web app)

`POST /auth/login`	Username + password → sets a session cookie
`POST /auth/logout`	Ends the session
`GET /auth/me`	Who am I logged in as? (the web app calls this on load)

There is deliberately **no sign-up endpoint**. Your account is created once with a command on the server (`python -m app.cli create-user`). A public sign-up page on a one-person system would only be an open door.

3.2 Devices (web app)

`POST /devices`	Create a device token for your laptop; shown *once*
`GET /devices`	List devices, with "last synced" time
`DELETE /devices/{id}`	Revoke a device (its token stops working immediately)

3.3 Sync (laptop)

`POST /sync`: the single round trip from Doc 06.

3.4 Analytics (web app)

`GET /analytics/summary`	Totals for a date range: commands, success rate, top apps
`GET /analytics/apps`	Time per app and per site, per day
`GET /analytics/commands`	Command history, newest first, filterable, in pages
`GET /analytics/failed`	Sentences that did not match a command (what to add next)
`GET /analytics/latency`	Timing breakdown per stage over time (Doc 04 budget)
`GET /analytics/calibration`	Confidence scores of right vs. wrong guesses (Doc 03 §11)

Concept: Pagination

A history of 20 000 commands is not sent in one response. `/analytics/commands` returns one **page** (say 50 rows) plus a marker for "the next page starts here". The web app asks for more as you scroll. Small responses keep both the free server and the browser light.

3.5 History edits (web app)

`PATCH /commands/{uuid}`	Mark right / wrong (the web review list)
`DELETE /commands/{uuid}`	Delete from history (tombstone, Doc 06)

Both are written to the change log, so the laptop applies them on its next sync, the same way web deletions already travel. These two fields (verdict and deletion) are the *only* parts of history the web app may change; everything else about a command is a fact recorded by the laptop.

3.6 Command library (web app)

`GET /library`	Latest snapshot of your command files, primitives and app names
`POST /library/edits`	Submit a change from the guided form (becomes a pending edit)
`GET /library/edits`	Pending edits and their status (waiting / applied / invalid / overridden)

3.7 Endpoints are code you write, not a fixed list

Nothing outside Listen decides which endpoints exist. Each endpoint is an ordinary Python function in `server/app/routes/`, labelled with the method and path it answers:

```
# server/app/routes/analytics.py
@router.get("/analytics/apps")
def app_usage(start: date, end: date, user=Depends(current_user)):
    return analytics_service.app_usage(user.id, start, end)
```

The `@router.get(...)` line is a **decorator** (the same idea as the primitive registry in Doc 03): it registers the function as the handler for `GET /analytics/apps`. FastAPI reads the parameter types (`start: date`) to validate the request, and `Depends(current_user)` makes FastAPI check the session cookie first and hand the function the logged-in user. Adding an endpoint means writing another function; renaming or removing one means editing the function. The tables above are the *starting* design.

Concept: API contract and versioning

The catch is your **clients**: the laptop agent and the web app call these exact paths and expect these exact shapes. Rename `/analytics/apps` without updating them and they break. So: **add freely, change carefully**. When other people depend on an API, the standard approach is **versioning**: endpoints live under `/v1/...`, and a breaking change is published as `/v2/...` *beside* the old one, so existing clients keep working until they move over. Listen starts at `/v1`, so that option is open from day one.

4 The PostgreSQL tables

Concept: PostgreSQL, and why not SQLite here too

PostgreSQL is a database *server*: a program that many connections can use at once, with stronger guarantees and richer types (such as `jsonb` for JSON you can search inside). The laptop has one program using its database, so SQLite's single file is ideal there. The server answers the laptop and the web app at the same time and runs on a host that may restart at any moment, so a real database server, which free hosts offer, is the right tool.

Table	Holds
`users`	id, username, **password hash** (never the password), created time
`sessions`	Logged-in browser sessions: hashed session id, user, expiry
`devices`	Your laptop(s): name, **token hash**, last sync time, revoked time
`failed_logins`	Recent failed login attempts per username (for lock-out)
`commands`	The same columns as the laptop's `commands` table, plus `device_id` and `received_at`
`usage_daily`	Same as the laptop's
`tombstones`	uuid + deletion time; uploads of these uuids are refused (Doc 06)
`change_log`	Numbered list of server-side changes the laptop must apply: deletions, verdicts, pending edits
`command_snapshots`	Latest command files (`jsonb`), their hash, primitives, app names
`pending_edits`	Edits from the web form: the edit, the hash it was based on, status, reason

Every table that holds your data has `user_id`, so adding more users later (the add-on from Doc 01) needs no table changes.

Concept: ORM and migrations on the server

On the server we use **SQLAlchemy**, an **ORM** (object-relational mapper): each table is described once as a Python class, and queries can be written in Python. With it comes **Alembic**, which generates the numbered migration scripts (Doc 05 §9) by comparing your classes to the real database. The laptop keeps plain SQL files: it has five simple tables and one user. Using the standard tool where each problem is bigger is a choice, not an inconsistency.

5 The ORM in depth: SQLAlchemy by example

5.1 The problem an ORM solves

Python thinks in **objects** (a `Command` with attributes `.heard`, `.result`); a database thinks in **tables and rows**. Without an ORM, your code constantly translates by hand: build an SQL string, send it, get back tuples like `(12, 'search what is an atom', 'executed', ...)`, and remember that position 2 is the result. An **object-relational mapper** does that translation for you.

Analogy

An ORM is an interpreter between two people who speak different languages. You speak Python ("give me my last 50 executed commands"); the database speaks SQL. The interpreter translates your sentence into SQL, and translates the database's answer (rows) back into Python objects. You can always ask the interpreter what it actually said, which is how you learn SQL along the way.

5.2 Step 1: describe a table once, as a class

```
from datetime import datetime
from sqlalchemy import ForeignKey
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class Command(Base):
```

```
    __tablename__ = "commands"                      # the table this class maps to

    id:      Mapped[int] = mapped_column(primary_key=True)
    uuid:    Mapped[str] = mapped_column(unique=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    ts:      Mapped[datetime]
    heard:   Mapped[str]
    result:  Mapped[str]
    verdict: Mapped[str | None]                     # "| None": may be empty
```

Each class is a table, each attribute is a column, and each *object* of the class is one row. `Mapped[str]` says "a text column that may not be empty"; `ForeignKey("users.id")` says "this number must be the id of a real user".

Database world	Python world (via the ORM)
Table `commands`	Class `Command`
One row	One `Command(...)` object
Column `heard`	Attribute `cmd.heard`
Foreign key to `users`	A link you can follow: `cmd.user`

5.3 Step 2: read, write, change, delete

All work happens inside a **session**: a conversation with the database that collects your changes and saves them together when you `commit()` (a transaction, Doc 05).

```
from sqlalchemy import select

with Session(engine) as session:
    # READ: my last 50 executed commands, newest first
    stmt = (select(Command)
            .where(Command.user_id == user.id, Command.result == "executed")
            .order_by(Command.ts.desc())
            .limit(50))
    for cmd in session.scalars(stmt):
        print(cmd.ts, cmd.heard)

    # CHANGE: mark one wrong. Just set the attribute.
    cmd = session.scalars(select(Command).where(Command.uuid == some_uuid)).one()
    cmd.verdict = "wrong"

    # CREATE: a new row is a new object
    session.add(Command(uuid=new_uuid, user_id=user.id, ts=now,
                        heard="volume 30", result="executed"))

    # DELETE
    session.delete(cmd)

    session.commit()        # everything above is saved together, or nothing is
```

For the READ, SQLAlchemy writes and sends this SQL for you:

```
SELECT commands.id, commands.uuid, commands.user_id, commands.ts, ...
FROM commands
WHERE commands.user_id = %(user_id)s AND commands.result = %(result)s
ORDER BY commands.ts DESC
LIMIT %(limit)s
```

Creating the engine with `echo=True` prints every SQL statement it sends. Turn it on while learning: you write Python, read the SQL it produced, and pick up SQL without having to learn it first.

5.4 Step 3: when you need a database-specific feature

The sync endpoint needs an **upsert** (Doc 06): "insert this row, or update it if the uuid already exists". PostgreSQL has a special command for that, and SQLAlchemy exposes it directly:

```
from sqlalchemy.dialects.postgresql import insert

stmt = insert(Command).values(rows)
stmt = stmt.on_conflict_do_update(
    index_elements=["uuid"],                          # "same uuid" means "same row"
    set_={"verdict": stmt.excluded.verdict, "result": stmt.excluded.result},
)
session.execute(stmt)
```

An ORM does not stop you from using the database's full power; it just makes the common cases short.

5.5 What an ORM gives you, and what it costs

Gives you	Costs you
Real Python objects and types; PyCharm autocompletes `cmd.` and flags typos	The SQL is hidden, so you can write slow queries without noticing (use `echo=True`)
Safety from **SQL injection** (below) by construction	One more library to learn on top of SQL
Alembic can generate migrations by comparing your classes to the database	The "N+1" trap: looping over 100 commands and touching `cmd.user` on each can send 101 queries instead of 1 (SQLAlchemy has options to load them together)

Concept: SQL injection

If code builds SQL by gluing text together, for example `"... WHERE heard = '" + text + "'"`, a value containing a quote can change the meaning of the whole statement; an attacker can turn "search" into "delete everything". The fix is **parameters**: the SQL is sent with placeholders (the `%(result)s` above) and the values separately, so a value can never be read as SQL. The ORM always uses parameters, which is one of the strongest reasons to use one.

6 Logging in without storing your password

Concept: Password hashing

The server stores a **hash** of your password, never the password itself. A password hash function (we use **Argon2**) turns a password into a fingerprint that cannot be turned back. At login, the server hashes what you typed and compares fingerprints. Two extra properties:

- a random **salt** is mixed in per user, so two identical passwords give different hashes and pre-computed tables of common passwords are useless;
- it is **deliberately slow** (a fraction of a second), which you never notice once, but which makes guessing millions of passwords impractical.

If the database ever leaked, the attacker would have fingerprints, not passwords.

Concept: Session cookie

After a successful login, the server creates a random **session id**, stores its hash in `sessions`, and sends the id to the browser in a **cookie**. The browser automatically attaches it to every later request, so you log in once, not on every click. The cookie is marked `HttpOnly` (page scripts cannot read it, so a malicious script cannot steal it), `Secure` (only sent over HTTPS) and `SameSite` (not sent when another website tries to make requests on your behalf). Sessions expire after 30 days; logging out deletes it.

Lock-out. After 5 failed logins for a username within 15 minutes, further attempts for that username are refused for 15 minutes. Failures are counted in the database, not in memory, because a free server that sleeps forgets its memory.

Device tokens. The laptop uses the token from `POST /devices`, sent in a header on every sync. Like passwords, the server keeps only a **hash** of each token, so a leaked database does not leak working tokens; and revoking a device sets `revoked_at`, which rejects its token at once.

Concept: Authentication vs. authorisation

Authentication: *who* are you? (password, session, device token). **Authorisation**: *what* may you touch? Every query is filtered by the caller's `user_id`, so even with multiple users, nobody can read another person's rows, by construction rather than by remembering to check.

7 Secrets and configuration

Every secret lives in **environment variables**, set on the host's dashboard in production and in a git-ignored `.env` file during development:

```
# server/.env  (never committed; server/.env.example lists the names with empty values)
DATABASE_URL=postgresql://...
SESSION_SECRET=...
WEB_APP_ORIGIN=https://...

# agent/.env   (never committed)
LISTEN_SERVER_URL=https://...
LISTEN_DEVICE_TOKEN=...
ANTHROPIC_API_KEY=...        # you paste this yourself; the AI builder reads it here
```

The server needs no AI key at all, because the AI builder runs on the laptop.

Concept: CORS

Browsers block a web page on one address from calling an API on another address unless the API explicitly allows it. That rule is **CORS** (cross-origin resource sharing). The server allows exactly one origin, your web app's address (`WEB_APP_ORIGIN`), and refuses every other website.

8 Living with a server that sleeps

Free hosts put an idle server to sleep and wake it on the next request, which then takes 20–50 seconds. The design absorbs this:

- **Laptop sync** uses a 60-second timeout. A request that still fails is treated as "offline": the outbox waits and the back-off from Doc 06 applies. Nothing is lost.
- **Web app** shows "Waking up the server…" instead of an error while the first request is slow.

- **No timed jobs on the server.** A sleeping server cannot run something "every night". Anything periodic happens *during* requests instead: for example, expired sessions and old login failures are cleaned up whenever someone logs in.

Watch out

Free tiers change their terms over time (limits, sleep rules, database size, expiry of free databases). The specific host is chosen in the deployment section, and the design does not depend on any one host: it only needs "a Python web server" and "a PostgreSQL database".

9 Errors and tests

One error shape. Every error response has the same JSON shape, so the laptop and the web app handle errors with one piece of code each:

```
{ "error": "device_revoked", "message": "This device token was revoked." }
```

Tests. `pytest` with FastAPI's test client, which sends fake HTTP requests straight to the app with no network. Tests run against a separate local PostgreSQL test database, created fresh for each run, so tests can never touch real data. Every endpoint gets tests for the happy path, the not-logged-in case (401) and invalid input (422), and the sync endpoint gets tests for duplicates, tombstoned uploads and batch limits (Doc 06).

10 Glossary

Argon2
A modern, deliberately slow password-hashing function.

Authentication / authorisation
Who you are / what you may access.

CORS
The browser rule controlling which websites may call an API.

Endpoint
One method + path an API answers.

Environment variable
A setting given to a program by its environment, used for secrets.

HTTP method / status code
What kind of request / whether it worked.

ORM
Library mapping database tables to Python classes (SQLAlchemy).

Pagination
Returning large lists one page at a time.

Pydantic model
A typed class that validates request and response data.

Salt Random data mixed into a password hash so identical passwords differ.

Session cookie
A random id the browser sends with every request after login.

11 Check your understanding

Check yourself

1. Why is there no sign-up endpoint?
2. The database leaks. What can the attacker do with `users` and `devices`?
3. Why is the login lock-out counted in the database instead of in memory?
4. What status code should `GET /analytics/apps` return if you are not logged in?
5. Why may the web app change a command's verdict but not its `heard` text?
6. The server is asleep when your laptop syncs. Walk through what happens.
7. Why does the server not need an Anthropic API key?